

One Codebase, Many Distributions

A harness humans and agents can both follow



122 distributions · one
repository

Jarek Potiuk Apache Airflow PMC

Amogh Rajesh Desai Airflow PMC · Astronomer

It's Friday. A CVE lands in boto3.

400 DAGs in production. Airflow 3.3.1.

```
pip install "apache-airflow==3.3.1" "apache-airflow-providers-amazon==9.34.0"
```

One provider moved.

- ✓ Scheduler untouched
- ✓ No DB migration
- ✓ No downtime
- ✓ No change request

Five years ago:

wait two months.

Two rooms

Inside the repo

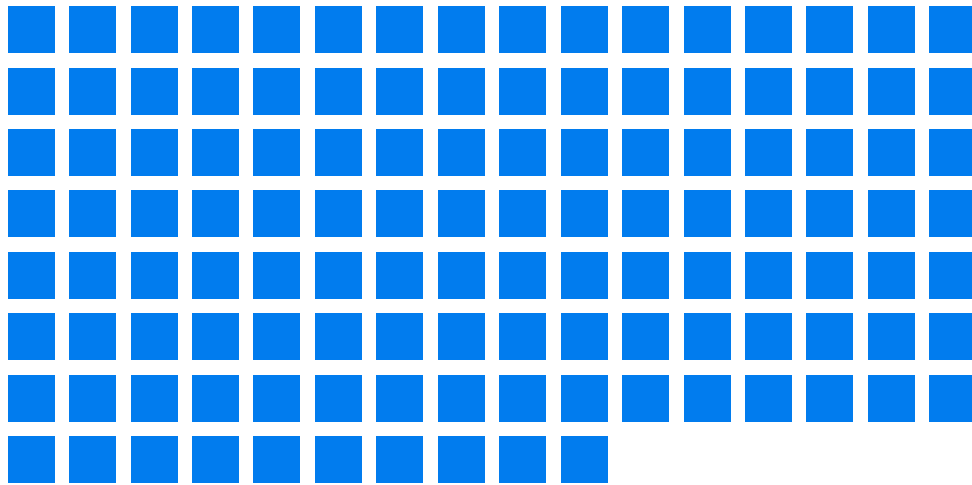
Jarek

Inside a cluster

Amogh

Same question, both rooms, every section.

The scale



122 distributions

one repository

1.2M

lines of Python

~103

providers

4,000+

contributors

700+

dependencies

170

prek hooks

~300

Opened PRs a week

~40

Opened PRs a day

~140

Merged PRs a week

~20

Merged PRs a day

<20

reviewers

"It's just another week, to be very honest."

Repo Organisation: Both answers are bad

Separate repos

- Version drift
- Integration pain
- No cross-cutting change

Giant monolith

- Leaky abstractions
- Implicit dependencies
- No individual versioning

"Eat the cake and have it too."

It did not start well

10,000+

lines of Bash

1

maintainer

Every

rebase, painful

The uv and prek harness

Two commands and one guardrail to hold 122 distributions together.

`uv sync`

`prek run`

`enforced boundaries`

Built so humans could navigate the repo.

The missing primitive

PEP 517

PEP 518

PEP 621

PEP 660

PEP 735

Standards, but no workspace.

uv workspaces

Many interconnected projects,
managed as one unit.

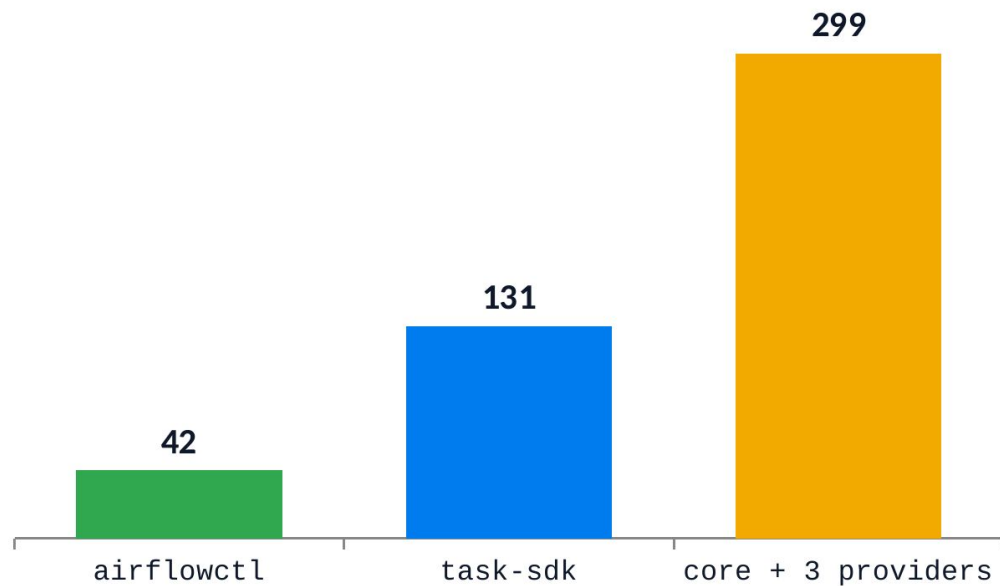
What you install

Distribution	Today	
<code>apache-airflow</code>	3.3.1	Meta-package · 125 extras
<code>apache-airflow-core</code>	3.3.1	Scheduler, API server, UI, migrations
<code>apache-airflow-task-sdk</code>	1.3.1	airflow.sdk · own 1.x line
<code>apache-airflow-ctl</code>	0.1.5	REST-only CLI · no DB access
<code>apache-airflow-providers-*</code>	SemVer	~103 of them · independent

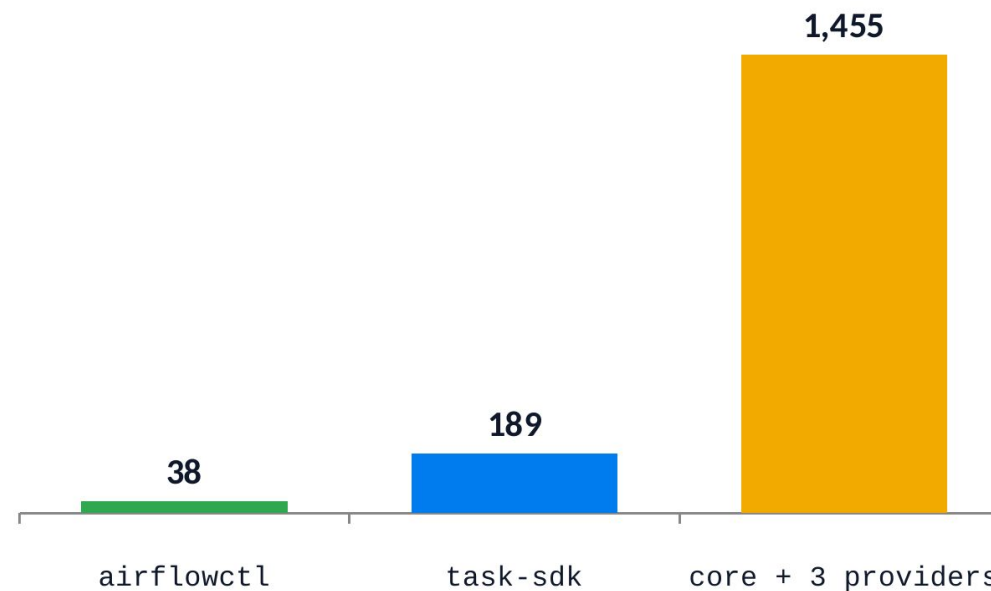
The versions don't match. That's the point.

What it costs to install

Packages



Megabytes



Megabytes in a clean Python 3.10 venv — our own measurements.

One command, any scope

The whole repo

```
$ uv sync
```

One component

```
$ cd airflow-core && uv sync
```

One provider

```
$ cd providers/amazon && uv sync
```

170+ hooks, every commit

Rust engine

drop-in for pre-commit

15+ configs

one per submodule

Seconds

not minutes

Tab completion

find the failing hook

Not all of them are style checks

Style

- Formatting
- Import order
- Spelling

Architecture

- Illegal imports
- Shared-code coupling
- Distribution boundaries

THE HARNESS

`uv sync`

`prek run`

`enforced boundaries`

`And this one.`

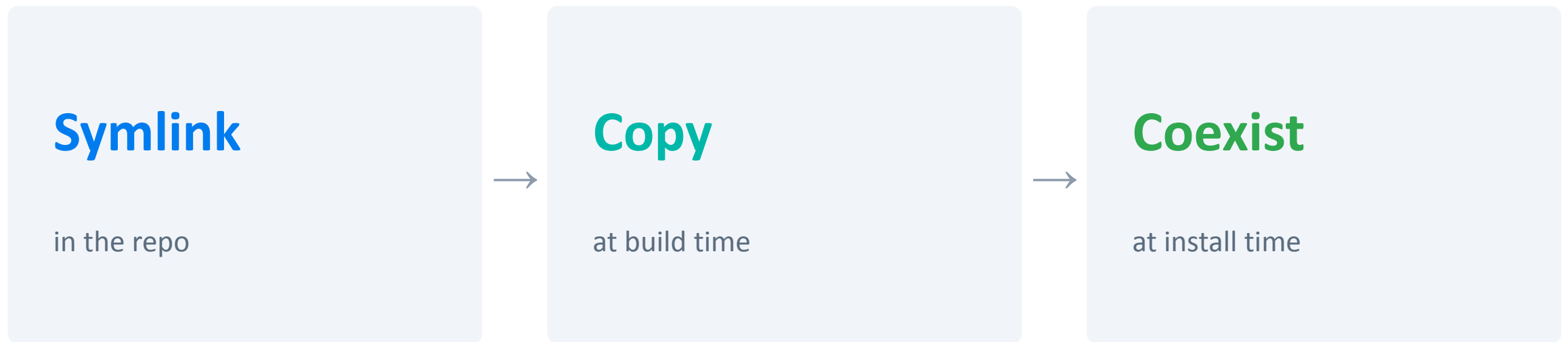
Shared libraries: the hard one

Python loads exactly

ONE version of a library.

Sharing → coupling → backwards compatibility forever.

Symlinks in, copies out



Each distribution carries the shared code it shipped with.

Control plane, execution plane



No database credentials in task code.

The harness meets the agents

Nobody designed it this way.

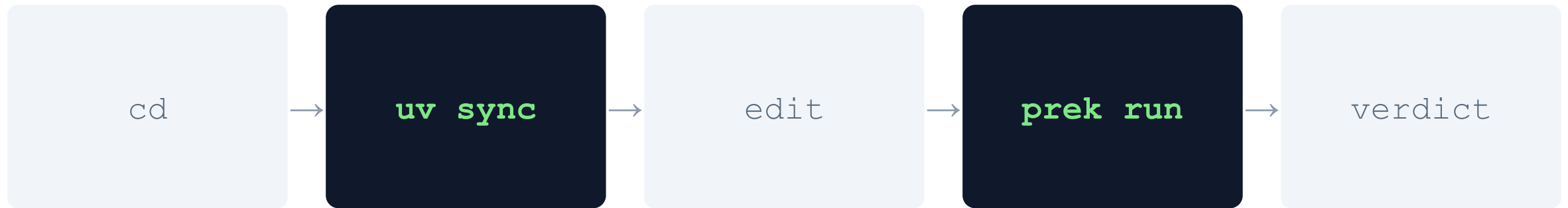
The harness is the spec

- 1 A scoped directory
- 2 A correct environment in one command
- 3 A verdict in one command
- 4 Boundaries the tool enforces

What a new contributor
needs.

What an agent needs.

The loop is the same either way



Two of the five steps are the harness.

A human runs it. An agent runs it. Nobody had to write it down twice.

THE HARNESS

uv sync

prek run

enforced boundaries

What the harness can tell a reviewer

It can

- The invariants hold
- The imports are legal
- No boundary was crossed

It can't

- Whether it's right
- The ripple effect
- Whether it's worth doing

Valid is not **correct**. That gap is **review**.

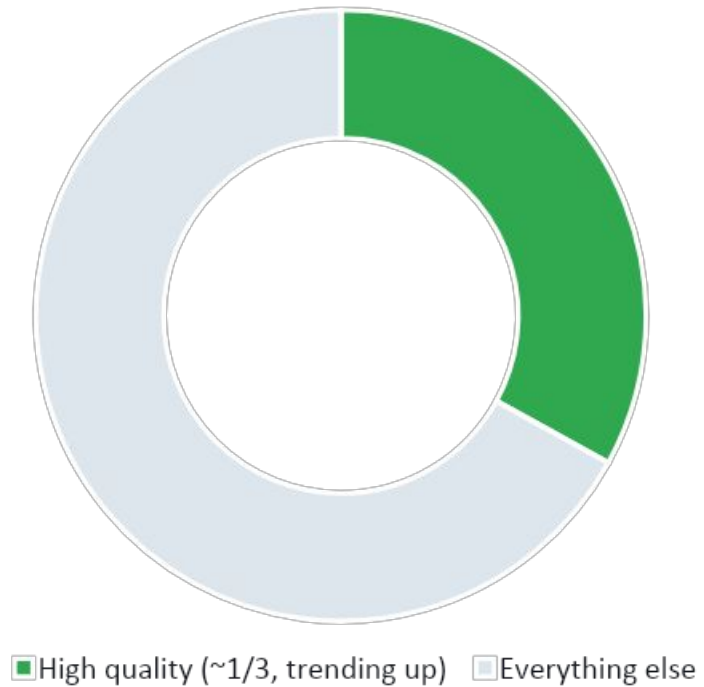
The failure mode isn't bad code

It's PRs that aren't reviewable.

The author doesn't have the context
the change requires.

Neither does the model.

The harness catches the mechanical part. The rest is a human.



Different rules for different modules

Providers

`contained blast radius`

New or agent-assisted

Harness is the gate

Experienced in that area

Merge on green

Shared libraries, SDK

`fan-out across distributions`

Owner review required

Compatibility check

Core, execution API, migrations

`institutional knowledge`

Talk to us first

Design discussion, then code

**A good harness is legible
to anything that can read.**

The scarce resource is no longer code.

It's review, and that is where we are stretched.

Steal this

Running Airflow

- Pin providers independently
- Use airflowctl
- Import from airflow.sdk

Running a monorepo

- uv workspaces
- prek
- Symlinks + hatchling
- Different rules per module

Learn more about monorepo



<https://s.apache.org/airflow-python-monorepo>

Jarek Potiuk · Amogh Rajesh Desai

Questions